PART 06–10 • PUBLISHING & DISTRIBUTION

Marketplace, Pages, Wiki, Releases & Packages

Actions ecosystem, static hosting, documentation, versioning, and artifact distribution

---

TOPICS COVERED

- Actions Marketplace Architecture
- Action Versioning & Supply Chain
- GitHub Pages Static Hosting
- SPA Frameworks on Pages
- Wiki Automation & Backup
- Annotated Tags & SBOM
- GitHub Packages (GHCR, npm, Maven, etc.)
- Enterprise Artifact Management
- Composite / Docker / JS Actions
- Pinned Versions & Security
- Jekyll & Custom Domains
- GitHub Wiki Internals
- Release Lifecycle
- Artifact Signing (Sigstore)
- OCI Registry
- Artifactory vs Nexus comparison

PART 6 — GitHub Actions Marketplace

6.1 Marketplace Architecture

The GitHub Actions Marketplace is a catalog of reusable workflow components. Actions on the Marketplace are simply GitHub repositories with a specific structure — any public repository with an action.yml (or action.yaml) at the root is eligible for listing.

Action Types

Type	Runtime	Startup	Platform	Best for
JavaScript	Node.js (on runner)	Fast	Any	API calls, logic, cross-platform
Composite	Runner shell	Fast	Any	Reusable step sequences
Docker container	Docker (container)	Slower	Linux only	Custom environments, CLI tools

Creating a JavaScript Action

```
# Directory structure:
my-action/
■■■ action.yml          # Action metadata
■■■ index.js            # Entry point
■■■ dist/               # Bundled output (commit this!)
■   ■■■ index.js
■■■ node_modules/       # (excluded from Marketplace; built into dist/)
■■■ package.json

# action.yml:
name: 'Deploy to Kubernetes'
description: 'Deploys a container image to Kubernetes'
author: 'Platform Team'
branding:
  icon: upload-cloud
  color: blue
inputs:
  image:
    description: 'Docker image to deploy'
    required: true
  namespace:
    description: 'Kubernetes namespace'
    required: false
    default: 'default'
outputs:
  deployment-url:
    description: 'URL of the deployed service'
runs:
  using: node20
  main: dist/index.js
  post: dist/cleanup.js  # Optional cleanup step

# index.js (entry point):
const core = require('@actions/core');
const exec = require('@actions/exec');
const github = require('@actions/github');

async function run() {
  try {
    const image = core.getInput('image', { required: true });
    const namespace = core.getInput('namespace');

    core.info(`Deploying ${image} to ${namespace}...`);
```

```
    await exec.exec('kubectl', [
      'set', 'image', 'deployment/api',
      `api=${image}`,
      `-n`, namespace
    ]);

    core.setOutput('deployment-url', `https://api.${namespace}.example.com`);
  } catch (error) {
    core.setFailed(error.message);
  }
}

run();
```

Action Versioning and Supply Chain Security

Actions are referenced by uses: owner/action@ref. The ref can be a branch, tag, or commit SHA.

```
# Reference patterns:
uses: actions/checkout@v4           # Tag (semver major)
uses: actions/checkout@v4.1.1       # Tag (full semver)
uses: actions/checkout@b4ffde65...  # Commit SHA (most secure)

# Security hierarchy (best to worst):
# 1. Commit SHA — immune to tag mutation attacks
# 2. Semver tag — vulnerable to tag deletion/recreation
# 3. Branch — dangerous (can change any time)
```

■ *In 2023, tj-actions/changed-files (used by 23,000+ repos) was compromised — the action was modified to exfiltrate secrets to a public log. Actions pinned to commit SHAs were unaffected. Always pin to SHA in production workflows.*

```
# Enterprise policy: pin all third-party actions to SHA
# Use Dependabot to automate SHA updates:
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: github-actions
    directory: /
    schedule:
      interval: weekly
    groups:
      github-actions:
        patterns: ["*"]

# Restrict allowed actions in org settings:
# Settings > Actions > Allow actions and reusable workflows:
# - Only local actions
# - Local + selected trusted owners (actions/, github/)
# - All actions (not recommended for enterprises)
```

6.2 Action Trust Model

GitHub offers three trust levels for Actions in organization settings:

- **Local only**: Only actions from the same org — maximum security
- **Trusted creators**: Add specific orgs/repos you trust (e.g., actions/, aws-actions/, docker/)
- **Any action**: All Marketplace actions — maximum risk

■ *Enterprise security teams should maintain an approved action allowlist and use policy-as-code to enforce it. See GitHub's 'Allowed actions' org policy.*

PART 7 — GitHub Pages

7.1 GitHub Pages Architecture

GitHub Pages serves static content from a repository branch or the /docs folder. Under the hood, GitHub Pages uses CDN (Fastly) to serve content globally with HTTPS via Let's Encrypt certificates.

Source	Branch/folder	Build	Use case
Classic	gh-pages branch	Jekyll or static	Docs, project sites
Actions	Any via deploy-pages	Any build tool	Modern apps, custom builds
Docs folder	/docs on main	Jekyll or static	Documentation alongside code

Deploying with GitHub Actions (Modern Approach)

```
# .github/workflows/pages.yml
name: Deploy GitHub Pages
on:
  push:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: pages
  cancel-in-progress: false

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm
      - run: npm ci
      - run: npm run build  # e.g. 'next export', 'ng build', 'vite build'
        env:
          NEXT_PUBLIC_BASE_PATH: /repo-name  # GitHub Pages subpath
      - uses: actions/upload-pages-artifact@v3
        with:
          path: ./out  # or ./dist, ./build, ./public

  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    steps:
      - uses: actions/deploy-pages@v4
        id: deployment
```

Framework Configurations for GitHub Pages

```
# Next.js static export (next.config.js):
/** @type {import('next').NextConfig} */
```

```
const nextConfig = {
  output: 'export',
  basePath: process.env.NEXT_PUBLIC_BASE_PATH || '',
  images: { unoptimized: true },  // Required for static export
};
export default nextConfig;

# Vite (vite.config.js):
export default {
  base: '/repo-name/',  // Set to '/' for custom domains
};

# MkDocs (mkdocs.yml):
site_name: 'Platform Engineering Docs'
theme:
  name: material
  features:
    - navigation.tabs
    - navigation.sections
    - search.suggest
plugins:
  - search
  - git-revision-date-localized

# Docusaurus (docusaurus.config.js):
const config = {
  url: 'https://myorg.github.io',
  baseUrl: '/my-docs/',
  organizationName: 'myorg',
  projectName: 'my-docs',
};
```

Custom Domains

```
# 1. Add CNAME file to repository root (or Pages source):
echo "docs.example.com" > CNAME

# 2. Configure DNS (at your DNS provider):
# For apex domain (example.com):
# A record:     185.199.108.153
# A record:     185.199.109.153
# A record:     185.199.110.153
# A record:     185.199.111.153
# AAAA records: 2606:50c0:8000::153 (etc.)

# For subdomain (docs.example.com):
# CNAME record: myorg.github.io

# 3. Enable HTTPS in Pages settings (auto via Let's Encrypt)
# 4. Enable 'Enforce HTTPS' after cert provisioning
```

Pages Limitations

Limit	Value	Notes
Site size	1 GB	Repository size also counts
Monthly bandwidth	100 GB	Soft limit, GitHub may contact you
Build timeout	10 minutes	Per build
Build frequency	10 builds/hour	Enforced limit
Server-side code	None	Static only — no SSR, no backend
Custom 404	404.html in root	Customize error pages

PART 8 — GitHub Wiki

8.1 Wiki Architecture

Every GitHub repository has an optional wiki, which is itself a separate Git repository stored at https://github.com/owner/repo.wiki.git. This means the wiki has full Git history, can be cloned, edited locally, and pushed. Pages are Markdown files (and optionally AsciiDoc, reStructuredText, MediaWiki, Textile, or Org-mode).

```
# Clone the wiki for local editing:
$ git clone https://github.com/myorg/myrepo.wiki.git
$ ls
Home.md
Architecture.md
API-Reference.md
_Sidebar.md    # Navigation sidebar
_Footer.md     # Footer content

# Edit locally and push:
$ vim Architecture.md
$ git add -A && git commit -m "Update architecture diagrams"
$ git push origin master  # Wiki uses 'master' branch
```

Wiki Automation

```
# Auto-generate API docs to wiki from workflow:
name: Update Wiki
on:
  push:
    branches: [main]
    paths: ['src/**/*.py', 'docs/api/**']

jobs:
  update-wiki:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Generate API documentation
        run: |
          pip install pdoc3
          pdoc --html --output-dir /tmp/api-docs src/
      - name: Update wiki
        uses: nicboul/wiki-action@v2
        with:
          repository: ${{ github.repository }}
          token: ${{ secrets.WIKI_TOKEN }}  # PAT with repo scope
          path: /tmp/api-docs/
```

PART 9 — GitHub Releases

9.1 Release Lifecycle

A GitHub Release is a deployment unit that packages a specific version of software with release notes, binary assets, and metadata. Releases build on Git's tag system but add GitHub-specific features.

```
# Full release automation workflow:
name: Release
on:
  push:
    tags:
      - 'v[0-9]+.[0-9]+.[0-9]+'  # Semantic version tags only

jobs:
  release:
    runs-on: ubuntu-latest
    permissions:
      contents: write
      id-token: write    # For OIDC signing
      packages: write    # For GHCR
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0  # Full history for changelog

      - name: Build binaries
        run: |
          GOOS=linux   GOARCH=amd64 go build -o dist/app-linux-amd64 .
          GOOS=darwin  GOARCH=arm64 go build -o dist/app-darwin-arm64 .
          GOOS=windows GOARCH=amd64 go build -o dist/app-windows-amd64.exe .

      - name: Generate SBOM
        uses: anchore/sbom-action@v0
        with:
          artifact-name: sbom.spdx.json
          output-file: dist/sbom.spdx.json

      - name: Sign binaries with Cosign (Sigstore)
        uses: sigstore/cosign-installer@v3
      - run: |
          for f in dist/*; do
            cosign sign-blob --yes --oidc-issuer https://token.actions.githubusercontent.com \
              --output-signature "${f}.sig" \
              --output-certificate "${f}.pem" \
              "$f"
          done

      - name: Create Release
        uses: softprops/action-gh-release@v2
        with:
          generate_release_notes: true  # Auto-generate from PR titles
          files: |
            dist/app-*
            dist/sbom.spdx.json
            dist/*.sig
            dist/*.pem
          body: |
            ## Installation
            Verify the signature:
            ```
            cosign verify-blob app-linux-amd64 \
              --signature app-linux-amd64.sig \
              --certificate app-linux-amd64.pem \
              --certificate-identity-regexp="github.com/myorg/myrepo" \
              --certificate-oidc-issuer="https://token.actions.githubusercontent.com"
            ```
```

Semantic Versioning in Releases

Version type	Example	When to use
Major (X.0.0)	2.0.0	Breaking API changes
Minor (x.Y.0)	1.3.0	New features, backwards-compatible
Patch (x.y.Z)	1.2.4	Bug fixes, security patches
Pre-release	2.0.0-beta.1	Testing releases
Build metadata	1.0.0+git.abc123	CI build identification

PART 10 — GitHub Packages

10.1 Package Registry Overview

GitHub Packages is an integrated package and container registry. It supports multiple package ecosystems and is tightly integrated with GitHub's permission model — the same GITHUB_TOKEN that authenticates Actions workflows can push and pull packages.

Registry	Protocol	Package manager	Example
Container (GHCR)	OCI / Docker	docker, podman, skopeo	ghcr.io/org/app:v1.2
npm	npm registry	npm, yarn, pnpm	@myorg/shared-lib@1.0.0
Maven	Maven repo	mvn, gradle	com.example:utils:1.0
NuGet	NuGet	dotnet nuget	MyOrg.Shared 1.0.0
PyPI	pip	pip, poetry	myorg-utils==1.0.0
RubyGems	gem	gem, bundler	myorg-sdk (1.0.0)

GHCR — GitHub Container Registry

```
# Authenticate:
echo "$GITHUB_TOKEN" | docker login ghcr.io -u $GITHUB_ACTOR --password-stdin

# Build and push:
docker build -t ghcr.io/myorg/api-service:v1.2.3 .
docker push ghcr.io/myorg/api-service:v1.2.3

# Multi-platform build:
docker buildx build \
  --platform linux/amd64,linux/arm64 \
  --push \
  -t ghcr.io/myorg/api-service:v1.2.3 .

# Inspect image:
docker manifest inspect ghcr.io/myorg/api-service:v1.2.3

# Sign with Cosign (attach to OCI registry):
cosign sign ghcr.io/myorg/api-service:v1.2.3

# Attach SBOM:
cosign attach sbom --sbom sbom.spdx.json ghcr.io/myorg/api-service:v1.2.3

# Verify signature:
cosign verify ghcr.io/myorg/api-service:v1.2.3 \
  --certificate-identity-regexp="github.com/myorg/api-service" \
  --certificate-oidc-issuer="https://token.actions.githubusercontent.com"
```

GitHub Packages vs Artifactory vs Nexus

Feature	GitHub Packages	JFrog Artifactory	Sonatype Nexus
Ecosystems	6 (GHCR, npm, Maven, NuGet, PyPI, Generic)	30+ (Universal)	Maven, npm, PyPI, Docker, etc.
GitHub integration	Native (GITHUB_TOKEN)	Via secrets/webhooks	Via secrets/webhooks
SSO/RBAC	GitHub RBAC	Enterprise SAML/LDAP	Enterprise SAML/LDAP

Feature	GitHub Packages	JFrog Artifactory	Sonatype Nexus
Dependency proxy	No	Yes (remote repos)	Yes (proxy repos)
High availability	GitHub SLA	HA clustering	HA clustering
On-premise	GHES only	Yes	Yes
Cost	Included with GitHub	Separate license	Separate license (OSS free)
Advanced scanning	Via GHAS	Xray	IQ Server
Verdict	Best for native GitHub workflows	Enterprise standard for large orgs	Popular OSS-first choice

■ *For enterprises already on GitHub Enterprise Cloud, GitHub Packages + GHCR provides excellent value with zero additional tooling. For orgs with complex multi-cloud, multi-VCS environments or needing proxying of upstream registries, Artifactory remains the gold standard.*

Package Retention and Cost

```
# GitHub Packages billing:
# Free tier per account:
# - GitHub Free: 500 MB storage, 1 GB bandwidth
# - GitHub Pro: 2 GB storage, 10 GB bandwidth
# - GitHub Team: 2 GB storage, 10 GB bandwidth
# - GitHub Enterprise: 50 GB storage, 50 GB bandwidth
# Overage: $0.25/GB storage, $0.50/GB bandwidth

# Retention policies (set per package/version):
# Via GitHub UI: Packages > Package settings > Danger Zone
# Or via Actions:
- uses: actions/delete-package-versions@v5
  with:
    package-name: my-container
    package-type: container
    min-versions-to-keep: 10
    delete-only-pre-release-versions: true
    ignore-versions: "^v1\.0\.0$"  # Always keep v1.0.0
```

Interview Questions — Marketplace, Pages, Packages

Q: Why should you pin GitHub Actions to commit SHAs instead of tags?

A: Tags are mutable — a malicious actor who compromises an action repository can delete and recreate a tag pointing to malicious code. Commit SHAs are immutable. If you pin to SHA, you get the exact code you audited. Dependabot can automatically update SHA pins with PRs when the action releases new versions.

Q: Explain the difference between GitHub Packages and GitHub Releases.

A: GitHub Releases package source code and binary assets (executables, installers) tied to a specific git tag with human-readable release notes. GitHub Packages hosts versioned package artifacts (Docker images, npm packages, etc.) consumable by build tools (docker pull, npm install). They complement each other: a release workflow can publish to both.

Q: How does GitHub Pages handle SPAs that use client-side routing?

A: GitHub Pages serves static files — a direct URL to /about will get a 404 because there's no about.html. The workaround is a custom 404.html that reads the URL and redirects to index.html with the path as a query parameter, which the SPA then reads and routes to the correct view. The 'spa-github-pages' project provides a standard implementation of this pattern.

Q: What is SBOM and why is it important in a release workflow?

A: A Software Bill of Materials (SBOM) is a formal, machine-readable inventory of all components in a software artifact — direct and transitive dependencies, versions, and their sources. It enables: vulnerability scanning against CVE databases, license compliance checks, supply chain auditing, and meeting regulatory requirements (US EO 14028 mandates SBOMs for federal software). Tools like syft and anchore/sbom-action generate SPDX or CycloneDX SBOMs from container images or source trees.